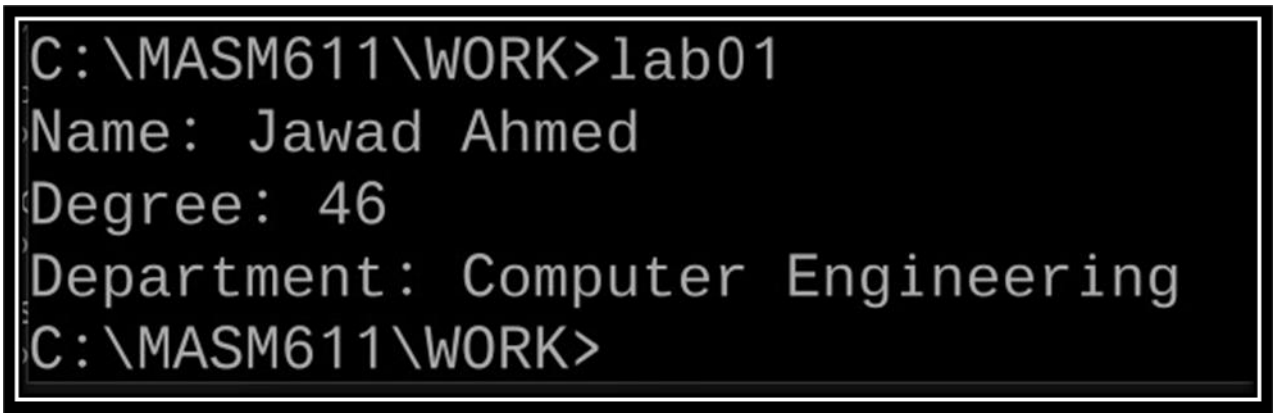
ASSIGNMENT

For the Course

EC-200 Data Structures



Group Members	
Jawad Ahmed	
Degree	46
CMS	533047
Syndicate	A
Submitted to	Prof. Anum Abdus Salam

DEPARTMENT OF COMPUTER ENGINEERING
NUST College of Electrical & Mechanical Engineering (CEME)

Assignment 01

❖ Table of Contents

❖ Objectives

The primary objectives of this assignment are:

- ❖ To design and implement a custom Doubly Linked List (DLL) Abstract Data Type in C++.
- ❖ To apply the DLL as the core backend for a dynamic music playlist manager.
- ❖ To master key playlist operations: insertion at any position, deletion by ID, title-based search, and bidirectional navigation.
- ❖ To integrate the SFML audio library for real-time .mp3 playback within a functional application.
- ❖ To apply Object-Oriented Programming principles by separating data structure, domain logic, and presentation into distinct classes.
- ❖ To optimize navigation performance to O(1) using a stored node pointer instead of an index-based traversal.
- ❖ To build a polished console UI using ANSI escape codes via the ConsoleUtils library.

❖ Introduction

A **Doubly Linked List (DLL)** is a linear, dynamic data structure in which each node holds a data element along with two pointers — *next* and *prev* — linking it to the node ahead and behind. Unlike a singly linked list, the DLL supports traversal in both directions, making it the ideal structure for a music player where a user must be able to skip forward and backward through a playlist.

This assignment implements **HIVE** — a lightweight, console-based music player. The playlist is managed entirely by a custom templated DLL. Real audio playback is handled by **SFML (Simple and Fast Multimedia Library)**, and the console interface uses ANSI escape codes via a *ConsoleUtils* library for colored output.

The application is structured into three layers that closely mirror professional software architecture:

1. DoublyLinkedList<T> — Generic data structure managing memory and node linkage.
2. Playlist — Domain logic wrapping the DLL with all required music player operations.
3. MusicPlayer — Presentation layer handling the console UI, SFML audio engine, and user input.

A key optimization implemented is **O(1) track navigation**: instead of storing a numeric index and traversing the list on every skip, the Playlist stores a direct node<Track>* pointer. Moving to the next or previous track is a single pointer dereference with no list traversal.

❖ Software

Software Used: Visual Studio Code.

Usage: The integrated development environment (IDE) was used to write, compile, and debug C++ code. The C++ compiler (MinGW/GCC) was utilized to manage dynamic memory allocation via new and delete keywords, allowing for the creation and destruction of nodes during runtime.

❖ Tasks

Task 1: Doubly Linked List ADT

Statement: Implement a custom templated Doubly Linked List supporting insertion at beginning, end, and any position; deletion from start, end, and any position; and O(1) node counting via a maintained size counter.

```
#pragma once
#include <iostream>

template <typename T>
struct node {
    T data;
    node<T>* next;
    node<T>* prev;
};

template <typename T>
class DoublyLinkedList {
private:
    node<T>* head;
    node<T>* tail;
    int listSize; // O(1)

    node<T>* getNewNode(T val) {
        node<T>* newNode = new node<T>;
        newNode->data = val;
        newNode->next = nullptr;
        newNode->prev = nullptr;
        return newNode;
    }

public:
    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
        listSize = 0;
    }

    ~DoublyLinkedList() {
        freeMemory();
    }

    node<T>* getHead() const {
        return head;
    }

    void insertAtBeginning(T val) {
```

```
        node<T>* newNode = getNewNode(val);

        if(head == nullptr) {
            head = tail = newNode;
        } else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }
        listSize++;
    }

    void insertAtEnd(T val) {
        node<T>* newNode = getNewNode(val);

        if(head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
        listSize++;
    }

    void insertAtAnyPos(int position, T val) {
        if(position <= 0 || position > listSize + 1) {
            std::cout << "Invalid position!\n";
            return;
        }

        if(position == 1) {
            insertAtBeginning(val);
            return;
        }

        if(position == listSize + 1) {
            insertAtEnd(val);
            return;
        }

        node<T>* newNode = getNewNode(val);
        node<T>* temp = head;

        for(int i = 1; i < position - 1; i++) {
            temp = temp->next;
        }
```

```
        newNode->next = temp->next;
        newNode->prev = temp;

        if(temp->next != nullptr) {
            temp->next->prev = newNode;
        }

        temp->next = newNode;
        listSize++;
    }

    void deleteFromStart() {
        if(isEmpty()) return;

        node<T>* temp = head;
        if(head == tail) {
            head = tail = nullptr;
        } else {
            head = head->next;
            head->prev = nullptr;
        }

        delete temp;
        listSize--;
    }

    void deleteFromEnd() {
        if(isEmpty()) return;

        node<T>* temp = tail;
        if(head == tail) {
            head = tail = nullptr;
        } else {
            tail = tail->prev;
            tail->next = nullptr;
        }

        delete temp;
        listSize--;
    }

    void deleteAtAnyPos(int position) {
        if(isEmpty()) {
            std::cout << "List is empty!\n";
            return;
        }
```

```
        if(position <= 0 || position > listSize) {
            std::cout << "Invalid position!\n";
            return;
        }

        if(position == 1) {
            deleteFromStart();
            return;
        }

        if(position == listSize) {
            deleteFromEnd();
            return;
        }

        node<T>* temp = head;
        for(int i = 1; i < position; i++) {
            temp = temp->next;
        }

        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;

        delete temp;
        listSize--;
    }

    // O(1) Complexity - No more loops!
    int nodeCount() const {
        return listSize;
    }

    bool isEmpty() const {
        return (head == nullptr);
    }

    void traverseForward() const {
        node<T>* temp = head;
        while(temp != nullptr) {
            std::cout << temp->data << "\n";
            temp = temp->next;
        }
    }

    void freeMemory() {
        node<T>* temp;
```

```
        while(head != nullptr) {
            temp = head;
            head = head->next;
            delete temp;
        }
        tail = nullptr;
        listSize = 0;
    }
};
```

Task 2: Playlist Class — All Required Operations

Statement: Implement the Playlist class wrapping DoublyLinkedList<Track>. Support add (beginning/end/position), remove by ID, search by title, display, navigate next/prev, and O(1) count.

CODE:

```
struct Track {
    int id;
    string title;
    string artist;
    int duration;
    string filePath;

    bool operator==(const Track& other) const { return id == other.id; }

    friend ostream& operator<<(ostream& os, const Track& t) {
        os << "[" << t.id << "] " << t.title << " by " << t.artist;
        return os;
    }
};

// 2. Playlist Class (Domain Layer)
class Playlist {
private:
    DoublyLinkedList<Track> dll;
    node<Track>* currentTrackNode;
    int nextId;

public:
    Playlist() : currentTrackNode(nullptr), nextId(1) {}

    void addTrack(const string& title, const string& artist, int duration, const string&
path) {
        Track newTrack = { nextId++, title, artist, duration, path };
        dll.insertAtEnd(newTrack);

        // If it's the first track, point current to it
```

```
        if (dll.nodeCount() == 1) {
            currentTrackNode = dll.getHead();
        }
    }

    void removeTrack(int id) {
        // Find position of ID (O(N) search)
        node<Track>* temp = dll.getHead();
        int pos = 1;
        while (temp != nullptr) {
            if (temp->data.id == id) {
                // Safety: If deleting the playing track, move pointer to next
                if (temp == currentTrackNode) {
                    moveNext();
                    if (temp == currentTrackNode) currentTrackNode = nullptr; // If it was
the only track
                }
                dll.deleteAtAnyPos(pos);
                return;
            }
            temp = temp->next;
            pos++;
        }
    }

    void moveNext() {
        if (currentTrackNode && currentTrackNode->next) {
            currentTrackNode = currentTrackNode->next;
        } else {
            currentTrackNode = dll.getHead(); // Loop back to start
        }
    }

    void movePrev() {
        if (currentTrackNode && currentTrackNode->prev) {
            currentTrackNode = currentTrackNode->prev;
        }
    }

    Track* getCurrentTrack() {
        if (currentTrackNode) return &(currentTrackNode->data);
        return nullptr;
    }

    void displayPlaylist() {
        dll.traverseForward();
    }
```

```
    int getTotalTracks() {
        return dll.nodeCount(); // This must be O(1)
    }
};
```

Task 3: SFML Audio Integration & Console UI

Statement: Implement the Playlist class wrapping DoublyLinkedList<Track>. Support add (beginning/end/position), remove by ID, search by title, display, navigate next/prev, and O(1) count.

CODE:

```
class MusicPlayer {
private:
    Playlist& playlist;
    sf::Music music;
    ConsoleUtils utils;
    bool isPlaying;

    void playAudio() {
        Track* current = playlist.getCurrentTrack();
        if (!current) return;

        music.stop();
        if (music.openFromFile(current->filePath)) {
            music.play();
            isPlaying = true;
        } else {
            isPlaying = false;
        }
    }

    void drawDashboard() {
        utils.clearConsole();

        // Header
        utils.setForegroundColor(ConsoleColor::BrightCyan);
        cout << "+-------------------------------------------------+\n";
        cout << "|                      HIVE                       |\n";
        cout << "+-------------------------------------------------+\n";

        // 2. Your Signature
        utils.setForegroundColor(ConsoleColor::BrightYellow);
        // 39 spaces ensures 'D' aligns perfectly with the box edge
        cout << "                                    BY: JAWAD AHMED\n\n";
```

```
        // Now Playing
        Track* current = playlist.getCurrentTrack();
        if (current) {
            utils.setForegroundColor(ConsoleColor::BrightGreen);
            cout << ">>> NOW PLAYING <<<\n";
            utils.setForegroundColor(ConsoleColor::White);
            cout << "Title  : " << current->title << "\n";
            cout << "Artist : " << current->artist << "\n";

            if (isPlaying) {
                utils.setForegroundColor(ConsoleColor::BrightYellow);
                cout << "Status : [ PLAYING ]\n\n";
            } else {
                utils.setForegroundColor(ConsoleColor::BrightRed);
                cout << "Status : [ PAUSED ]\n\n";
            }
        } else {
            utils.setForegroundColor(ConsoleColor::BrightRed);
            cout << ">>> PLAYLIST EMPTY <<<\n\n";
        }

        // Playlist Overview
        utils.setForegroundColor(ConsoleColor::BrightMagenta);
        cout << "--- PLAYLIST (" << playlist.getTotalTracks() << " Tracks) ---\n";
        utils.setForegroundColor(ConsoleColor::White);
        playlist.displayPlaylist();
        cout << "\n";

        // Controls
        utils.setForegroundColor(ConsoleColor::BrightCyan);
        cout << "+-----------------------------------------------+\n";
        utils.setForegroundColor(ConsoleColor::White);
        cout << "[1] Play/Pause    [2] Next Track    [3] Prev Track\n";
        cout << "[4] Add Song      [5] Remove Song   [6] Exit\n";
        cout << "+-----------------------------------------------+\n";
        utils.setForegroundColor(ConsoleColor::BrightGreen);
        cout << "Enter choice: ";
        utils.setDefaultColor();
    }

    void handleInput(int choice) {
        switch (choice) {
            case 1: // Play/Pause
                if (isPlaying) {
                    // If currently playing, simply pause
                    music.pause();
                    isPlaying = false;
```

```
                } else {
                    // CHECK: Is the song actually loaded?
                    // If Duration is 0, it means no file is loaded yet (Fresh Start).
                    if (music.getDuration() == sf::Time::Zero) {
                        playAudio(); // Load the file and start from scratch
                    } else {
                        // File is loaded, just resume from where we left off
                        music.play();
                        isPlaying = true;
                    }
                }
                break;
            case 2: // Next
                playlist.moveNext();
                playAudio();
                break;
            case 3: // Prev
                playlist.movePrev();
                playAudio();
                break;
            case 4: { // Add
                string title, artist, path;
                cout << "Enter Title: "; cin.ignore(); getline(cin, title);
                cout << "Enter Artist: "; getline(cin, artist);
                cout << "Enter Filepath (.mp3): "; getline(cin, path);
                playlist.addTrack(title, artist, 0, path);
                break;
            }
            case 5: { // Remove
                int id;
                cout << "Enter Track ID to delete: ";
                cin >> id;
                playlist.removeTrack(id);
                // Resync audio in case we deleted the currently playing track
                if (playlist.getCurrentTrack() == nullptr) music.stop();
                break;
            }
            default:
                break;
        }
    }

public:
    MusicPlayer(Playlist& p) : playlist(p), isPlaying(false) {
        utils.enableVirtualTerminal();
    }
```

```
    void run() {
        bool running = true;
        int choice;

        while (running) {
            drawDashboard();

            cin >> choice;

            if (cin.fail()) {
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                continue;
            }

            if (choice == 6) {
                running = false;
                continue;
            }

            handleInput(choice);
        }

        utils.clearConsole();
        utils.setForegroundColor(ConsoleColor::BrightCyan);
        utils.showCredits();
    }
};
```

OUTPUT:

```
+-----------------------------------------------+
|                     HIVE                      |
+-----------------------------------------------+
                                  BY: JAWAD AHMED

>>> NOW PLAYING <<<
Title  : Faslon Ko Takkaluf
Artist : Atif Aslam
Status : [ PAUSED ]

--- PLAYLIST (3 Tracks) ---
[1] Faslon Ko Takkaluf by Atif Aslam
[2] Balaghal Ula Bi Kamaalihi by Ali Zafar
[3] MUSTAFA JAAN E REHMAT by Atif Aslam

+-----------------------------------------------+
[1] Play/Pause    [2] Next Track    [3] Prev Track
[4] Add Song      [5] Remove Song   [6] Exit
+-----------------------------------------------+
Enter choice:
```

WHEN PRESSED 1:

```
+-----------------------------------------------+
|                     HIVE                      |
+-----------------------------------------------+
                                  BY: JAWAD AHMED

>>> NOW PLAYING <<<
Title  : Faslon Ko Takkaluf
Artist : Atif Aslam
Status : [ PLAYING ]

--- PLAYLIST (3 Tracks) ---
[1] Faslon Ko Takkaluf by Atif Aslam
[2] Balaghal Ula Bi Kamaalihi by Ali Zafar
[3] MUSTAFA JAAN E REHMAT by Atif Aslam

+-----------------------------------------------+
[1] Play/Pause    [2] Next Track    [3] Prev Track
[4] Add Song      [5] Remove Song   [6] Exit
+-----------------------------------------------+
Enter choice:
```

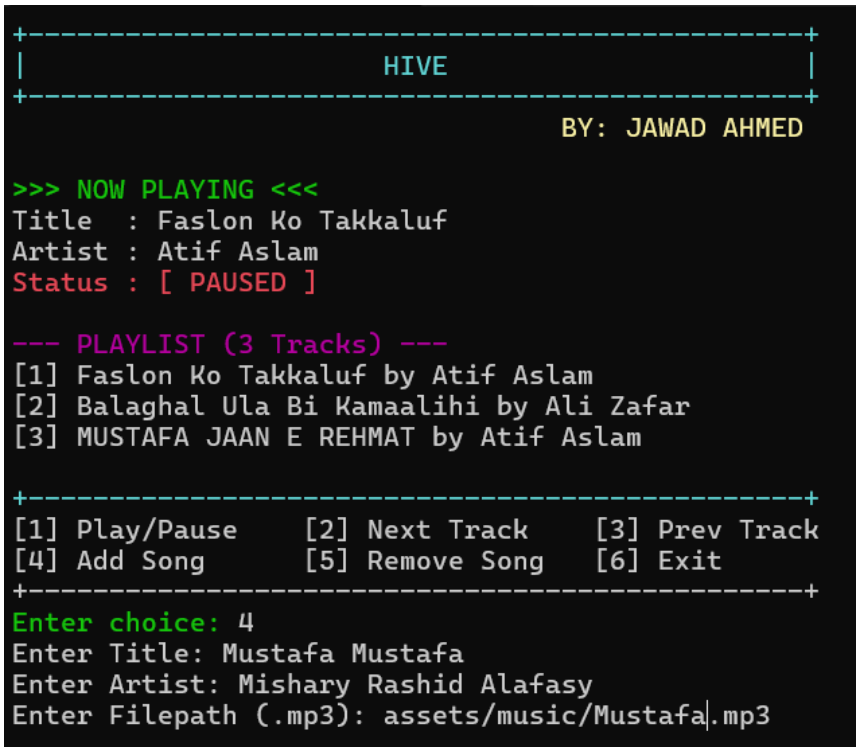
WHEN PRESSED 2:

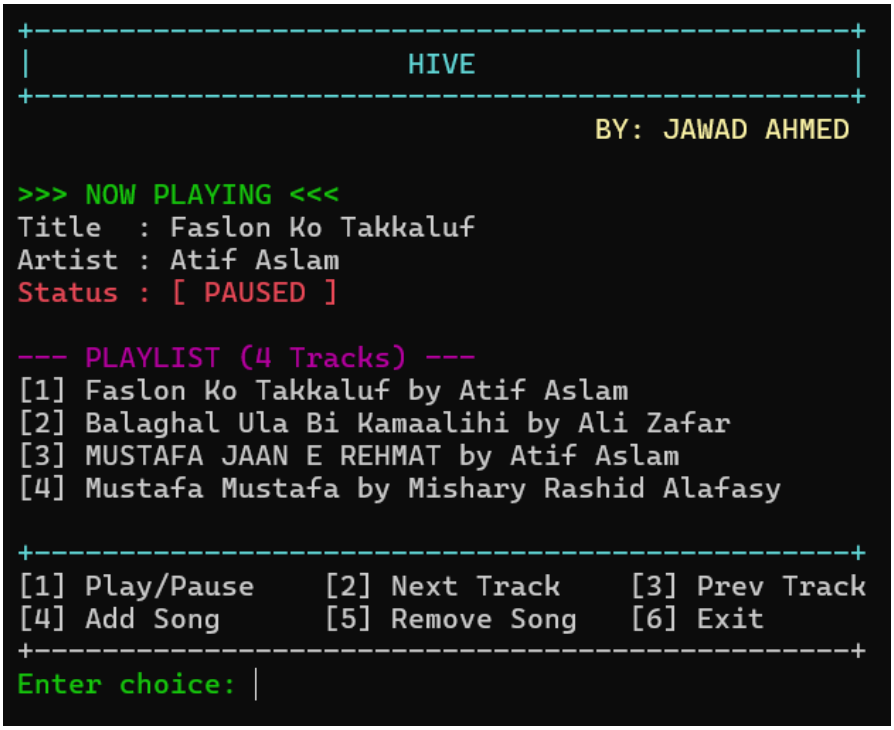
```
+------------------------------------------------+
|                      HIVE                      |
+------------------------------------------------+
                                  BY: JAWAD AHMED

>>> NOW PLAYING <<<
Title  : Balaghal Ula Bi Kamaalihi
Artist : Ali Zafar
Status : [ PLAYING ]

--- PLAYLIST (3 Tracks) ---
[1] Faslon Ko Takkaluf by Atif Aslam
[2] Balaghal Ula Bi Kamaalihi by Ali Zafar
[3] MUSTAFA JAAN E REHMAT by Atif Aslam

+------------------------------------------------+
[1] Play/Pause    [2] Next Track    [3] Prev Track
[4] Add Song      [5] Remove Song   [6] Exit
+------------------------------------------------+
Enter choice:
```

WHEN PRESSED 3:

```
+------------------------------------------------+
|                      HIVE                      |
+------------------------------------------------+
                                  BY: JAWAD AHMED

>>> NOW PLAYING <<<
Title  : Faslon Ko Takkaluf
Artist : Atif Aslam
Status : [ PLAYING ]

--- PLAYLIST (3 Tracks) ---
[1] Faslon Ko Takkaluf by Atif Aslam
[2] Balaghal Ula Bi Kamaalihi by Ali Zafar
[3] MUSTAFA JAAN E REHMAT by Atif Aslam

+------------------------------------------------+
[1] Play/Pause    [2] Next Track    [3] Prev Track
[4] Add Song      [5] Remove Song   [6] Exit
+------------------------------------------------+
Enter choice:
```

WHEN PRESSED 4:





WHEN PRESSED 5:

```
+-----------------------------------------------+
|                    HIVE                       |
+-----------------------------------------------+
                                  BY: JAWAD AHMED

>>> NOW PLAYING <<<
Title  : Faslon Ko Takkaluf
Artist : Atif Aslam
Status : [ PAUSED ]

--- PLAYLIST (3 Tracks) ---
[1] Faslon Ko Takkaluf by Atif Aslam
[2] Balaghal Ula Bi Kamaalihi by Ali Zafar
[3] MUSTAFA JAAN E REHMAT by Atif Aslam

+-----------------------------------------------+
[1] Play/Pause    [2] Next Track    [3] Prev Track
[4] Add Song      [5] Remove Song   [6] Exit
+-----------------------------------------------+
Enter choice:
```

WHEN PRESSED 6:

```
Default Credits:
---------------
Created by: Jawad Ahmed
Version: 1.0
```

- Answers

1. **Why is a Doubly Linked List preferred over an array for this playlist?**
   An array requires a fixed size at declaration, meaning it must be resized (reallocated and copied) every time a track is added beyond capacity — an O(n) operation. Mid-list insertions and deletions also require shifting all subsequent elements, again O(n). The Doubly Linked List has no fixed capacity; each node is allocated independently on the heap. Insertions and deletions at any position only require pointer rewiring — O(1) once the target node is located — with no data shifting.

2. **Why is a Doubly Linked List preferred over a Singly Linked List?**
   A Singly Linked List only has a *next* pointer, so navigating backward to the previous track requires traversing from the head each time — O(n). A music player's most frequent operation is skipping forward and backward between tracks. The DLL's *prev* pointer enables movePrev() in O(1), making bidirectional navigation seamless and efficient.

3. **What is the O(1) navigation optimization and why does it matter?**
   A standard implementation stores a numeric index (e.g. currentIndex = 2) and calls getNthNode(currentIndex) on every track skip, which traverses from the head each time — O(n) per navigation event. In HIVE, the Playlist class stores a direct node<Track>* pointer called currentTrackNode. Moving to the next or previous track is a single pointer dereference — O(1) — with no list traversal required. Since navigation is the most frequent operation in a music player, this optimization has the most practical impact.

❖ Conclusion

Through this assignment, I successfully designed and implemented HIVE — a complete, functional music player backed by a custom Doubly Linked List data structure. The exercise demonstrated that the choice of data structure is not academic; it directly determines what operations are efficient or prohibitively expensive for a given application.

The DLL proved to be the ideal choice: it handled dynamic playlist sizing without reallocation, supported mid-list insertions and deletions via O(1) pointer rewiring, and enabled bidirectional navigation through the prev pointer — a capability entirely absent in a singly linked list or array.

Two key optimizations were implemented beyond the basic requirements. First, a **maintained listSize counter** in the DLL reduced getTotalTracks() from O(n) to O(1). Second, storing a direct node<Track>* pointer in the Playlist class reduced track navigation from O(n) to O(1) — the single most impactful optimization for a music player application.

Integrating SFML demonstrated how a well-designed backend data structure can be cleanly connected to a real-world feature like audio playback. The three-class OOP architecture — separating data, logic, and presentation — ensured each component remained independently understandable and maintainable, reflecting professional software design principles.